
PROJECT BIG DATA YELP SENTIMENT PIPELINE

Analyse des sentiments

Réalisé par :

Boudersa Kaouther

Arif Bouchera

Bouaineh Salsabil

Rapport d'architecture Big Data
Pipeline complet d'analyse de sentiments

Vue d'Ensemble du Pipeline

Analyser des millions d'avis Yelp pour prédire automatiquement leur tonalité (positif, neutre, négatif), en utilisant des technologies "Big Data" (Hadoop, Spark)

Source	Data
1	Yelp Dataset JSON (3.45 GB - 6.96M reviews)
2	HDFS Ingestion : <code>hdfs dfs -put /data/yelp/raw/</code> (Stockage distribué)
3	Spark Processing : PySpark - Tokenization TF-IDF - Feature Eng.
4	ML Training (3 models) : LogReg / RF / NB - 250K reviews sample
5	MongoDB Storage : 5000 predictions - Model metrics stored

TABLE 1 – Étapes du pipeline

Infrastructure Docker — Conteneurs

un environnement avec 7 conteneurs Docker interconnectés

Chaque service a un rôle précis :

HDFS (Namenode + 3 Datanodes) : Le système de stockage distribué. Il découpe le fichier JSON en blocs et les réplique pour plus de sécurité.

Spark (Master + Worker) Le moteur de calcul distribué. C'est le cerveau qui va lire les données, les nettoyer, les transformer et entraîner les modèles de Machine Learning.

Jupyter : environnement de développement interactif

MongoDB : La base de données finale qui stocke les prédictions et les performances des modèles.

HDFS — Système de Fichiers Distribué

Caractéristique	Détail
Découpage	Le fichier 3.45 GB est découpé en 26 blocs de 128 MB chacun
Réplication	Chaque bloc est copié sur 3 DataNodes
Santé	<code>hdfs fsck</code> → HEALTHY, 0 blocs corrompus, 0 blocs manquants
Répertoires	<code>/data/yelp/raw</code> - <code>/data/yelp/processed</code> - <code>/data/yelp/features</code> - <code>/models</code>

Listing 1 – Commandes HDFS utilisées

```
hdfs dfs -mkdir -p /data/yelp/raw
hdfs dfs -mkdir -p /data/yelp/processed
hdfs dfs -mkdir -p /data/yelp/features
hdfs dfs -mkdir -p /models
```

```
# Copier le fichier JSON dans HDFS
```

```
hdfs dfs -cp /yelp/raw/yelp_academic_dataset_review.json /data/yelp/
raw/
hdfs dfs -chmod -R 777 /data # donner les droits d'accès
```

Flux de Données Spark HDFS MongoDB

1. Lecture HDFS (Stockage distribué)

*Le fichier Yelp (3.45 GB) est trop grand pour une seule machine

*HDFS le découpe en 26 blocs de 128 MB chacun

*Chaque bloc est copié sur 3 DataNodes (réplication = sécurité, si un DataNode tombe les données restent)

*Le Namenode est le "chef" — il sait où est chaque bloc mais ne stocke rien lui-même

*Résultat : 3.45 GB répartis physiquement sur 3 machines, accessibles comme un seul fichier

```
spark = SparkSession.builder.appName("YelpSentiment").getOrCreate()
df = spark.read.json("hdfs://namenode:9000/data/yelp/raw/*.json")
```

2. Spark — Traitement des données

*Spark lit les 26 blocs HDFS en parallèle via le Master

*Le Master divise le travail entre worker1 et worker2 (8 cores au total)

*Spark charge les données en RAM (pas sur disque) → très rapide

*À l'action (pipeline.fit()), tout s'exécute en parallèle sur les 2 workers

3. Pipeline ML — Traitement du texte

Tokenizer → découpe chaque review en mots individuels

StopWordsRemover → supprime les mots inutiles (the, a, is...)

HashingTF → transforme chaque mot en nombre (vecteur de fréquences)

IDF → pondère les mots rares plus importants que les mots fréquents

Résultat : chaque review = un vecteur de chiffres que le modèle peut comprendre

4. Entraînement

3 modèles entraînés sur 80% des données (199 815 reviews) 20% test (50 185 reviews).

Logistic Regression (F1=0.82, Accuracy=83.2)

Random Forest (F1=0.58, Accuracy=70.3)

Naive Bayes (F1=0.77, Accuracy=74.9)

Les calculs se font en parallèle sur worker1 + worker2 simultanément

Logistic Regression

83.2%

Meilleur modèle

6. MongoDB — Stockage des résultats

pymongo écrit chaque prédiction dans la collection `reviewspredictions`

Chaque document stocke : `reviewid, predictedclass, probability, modelused, processedat`

Les métriques des 3 modèles sont stockées dans `modelmetrics`

Total : 5000 prédictions + 3 métriques sauvegardées dans la DB `yelpsentiment`

Stack Technologique Complète

Tableau détaillé de chaque outil utilisé dans le pipeline, son rôle précis et les métriques observées lors de l'exécution réelle du projet.

Big Data Storage		
Apache HDFS 3.x	Hadoop	Système de fichiers distribué. Stocke le dataset Yelp brut (3.45 GB). 26 blocs × 128 MB, réplication sur 3 Data-Nodes. Format : JSON ligne par ligne.
MongoDB 6.0		Base NoSQL orientée documents. Stocke 5 000 prédictions + 3 métriques de modèles dans la DB <code>yelp_sentiment</code> . Collections : <code>reviews_predictions</code> , <code>model_metrics</code> .
Traitement & ML		
Apache Spark (PySpark)		Moteur de calcul distribué en mémoire. Lit le JSON depuis HDFS, applique les transformations (nettoyage, tokenisation, TF-IDF) et entraîne les modèles ML via MLlib.
Spark MLlib		Bibliothèque ML distribuée. Modèles utilisés : Logistic Regression (acc. 83.2%), Random Forest (acc. 70.3%), Naïve Bayes (acc. 74.9%). Dataset : 250 000 reviews.
scikit-learn / pandas		Utilisés en post-traitement. Jupyter pour validation des métriques et analyse complémentaire des résultats de prédiction.
Infrastructure & Orchestration		
Docker & Docker Compose		Conteneurisation de tous les services. Réseau bridge <code>docker_{default}</code> (172.20.0.0/16). 7 conteneurs orchestrés : namenode, 2 datanodes, spark-master, spark-worker, jupyter, mongoddb.
Jupyter Lab		Interface interactive (port 8888) pour le développement du pipeline. Authentication par token. Connecté à Spark via <code>SparkSession</code> et à MongoDB via <code>pymongo</code> .
Langages & Bibliothèques		
Python 3.x		Langage principal du pipeline. Dépendances : <code>pyspark</code> , <code>pymongo</code> , <code>pandas</code> , <code>scikit-learn</code> , <code>numpy</code> , <code>matplotlib</code> .
HDFS CLI / Shell		Commandes <code>hdfs dfs</code> utilisées pour créer les répertoires, copier les fichiers, vérifier l'état (<code>fsck</code>) et gérer les permissions (<code>chmod 777</code>).

TABLE 2 – Stack technologique complet